

Hands-On Workshop: Linux seccomp Isolation

Buildroot 2025.02 Environment, QEMU Emulation, and strace Diagnosis

1. Workshop Overview & Objectives

This hands-on workshop teaches embedded systems engineers the fundamentals of kernel-level sandbox isolation using Linux Secure Computing Mode (seccomp). Engineers will construct applications on an Ubuntu host environment, embed them within a custom x86_64 Linux system image using Buildroot rootfs overlays, and boot the target in a QEMU emulator. A critical debugging component includes utilizing strace inside the target to dissect and visualize the exact system calls blocked by seccomp.

Key Learning Outcomes:

- Understand seccomp strict mode (Mode 1) vs. filter mode (Mode 2) via libseccomp.
- Implement programmatic system call filtering using explicit allowlists.
- Utilize strace inside an embedded rootfs to analyze sandbox violations and identify missing dependencies in rulesets.
- Inject host binaries into a target file system via Buildroot rootfs overlays.

2. Development Environment Setup

Step 2.1: Install Host Prerequisites

```
sudo apt-get update
sudo apt-get install -y build-essential libseccomp-dev qemu-system-x86 cpio
unzip rsync bc python3
```

Step 2.2: Configure Buildroot & Rootfs Overlays

- Create a dedicated workspace directory on your host:

```
mkdir -p ~/seccomp_workshop/overlay/usr/bin
```

- Download and extract Buildroot 2025.02, then launch the configuration menu:

```
cd ~/seccomp_workshop
wget https://buildroot.org/downloads/buildroot-2025.02.tar.gz
tar -xf buildroot-2025.02.tar.gz
cd buildroot-2025.02
make qemu_x86_64_defconfig
make menuconfig
```

- Inside the configuration menu, apply the following settings:
 - Navigate to: System configuration ---> Root filesystem overlay directories
 - Enter the absolute path: /home/<your_username>/seccomp_workshop/overlay
 - Navigate to: Target packages ---> Libraries ---> Security ---> [*] libseccomp
 - Navigate to: Target packages ---> Debugging, profiling and benchmark ---> [*] strace
 - Save your configuration and exit.

3. Core Diagnostic Tools: What is strace?

strace (System Call Trace) is a diagnostic utility for Linux that monitors the intersection between user-space applications and the Linux kernel. Because user processes execute in an isolated environment (User Mode/Ring 3), they cannot access hardware directly. Any resource request (such as file I/O, process handling, or querying the system time) requires generating a software interrupt via a system call to hand execution over to Kernel Mode (Ring 0).

By relying on the ptrace() kernel infrastructure, strace intercepts these transitions twice per call: once upon entry to extract the syscall arguments from CPU registers, and once upon exit to log the return code. When validating seccomp sandboxes, strace translates a silent termination into highly readable debug metadata, indicating exactly which system call triggered the kernel isolation mechanism.

4. Hands-On Assignments

Exercise 1: Enforcing Strict seccomp Mode (Mode 1)

Source Code Layout (strict_seccomp.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/prctl.h>
#include <linux/seccomp.h>
#include <fcntl.h>

int main() {
    printf("Step 1: Application initialization. System calls
    unrestricted.\n");

    if (prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT) < 0) {
        perror("prctl SECCOMP_MODE_STRICT failed");
        return 1;
    }
    printf("Step 2: Strict mode activated. Authorized write() execution.\n");

    int fd = open("/etc/hostname", O_RDONLY);
    printf("Step 3: This message will not be seen if blocked. FD: %d\n", fd);
```

```
    return 0;
}
```

Compilation and Deployment Pipeline:

```
gcc -static strict_seccomp.c -o
~/seccomp_workshop/overlay/usr/bin/strict_seccomp
```

Exercise 2: High-Level Abstraction and Allowlists via libseccomp

Source Code Layout (lib_seccomp.c):

```
#include <stdio.h>
#include <unistd.h>
#include <seccomp.h>
#include <sys/types.h>

int main() {
    scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_KILL);

    seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0);
    seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(exit_group), 0);
    seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(fstat), 0);

    printf("Loading seccomp filter...\n");
    seccomp_load(ctx);

    printf("This write() is allowed.\n");

    printf("Attempting getpid() which is NOT whitelisted...\n");
    getpid();

    printf("This will not print.\n");
    seccomp_release(ctx);
    return 0;
}
```

Compilation and Deployment Pipeline:

```
gcc lib_seccomp.c -o ~/seccomp_workshop/overlay/usr/bin/lib_seccomp -lseccomp
```

5. System Build and Execution Pipeline

```
cd ~/seccomp_workshop/buildroot-2025.02
make
```

Booting the Image in QEMU

```
qemu-system-x86_64 -M pc -kernel output/images/bzImage -drive  
file=output/images/rootfs.ext2,if=virtio,format=raw -append "root=/dev/vda  
console=ttyS0" -nographic
```

6. Target Execution, Diagnostics & strace Verification

Step 6.1: Analyzing Strict Mode with strace

```
/usr/bin/strict_seccomp  
strace /usr/bin/strict_seccomp
```

Observation Task: Look closely at the strace output. Engineers should identify that the program successfully traces up until the `prctl()` call and the trailing allowed `write()`. The second it targets `open()`, strace intercepts a '+++ killed by SIGSYS (bad system call) +++' status message.

Step 6.2: Tracking the libseccomp Allowlist Violation

```
strace /usr/bin/lib_seccomp
```

Observation Task: Engineers will notice that before `getpid()` is executed, multiple system calls (such as `openat`, `mmap`, `brk`) are processed by the dynamic linker. Because they occur BEFORE `seccomp_load()` is reached, the process survives. Once `seccomp_load()` passes, call `getpid()` and verify that strace immediately catches the kernel SIGSYS termination signal.